



Meowtion

On-device AI that reads your cat's health from its daily habits.

Meowtion Technical Documentation

On-device behaviour recognition for early feline health insight

Jerome Graves

Revision 1.0 • June 2026

Abstract. Meowtion is a smart cat collar that infers a cat's health from changes in its daily habits. A battery-powered collar classifies behaviour (eating, drinking, activity, rest, purring) on the device itself, relays the result over Bluetooth Low Energy to a mains-powered base station, and the station forwards it to the cloud for a live owner-facing dashboard. The behaviour set is not hard-coded: owners define the actions to recognise, label captured clips, and a cloud trainer produces tiny quantised models that are delivered back to the collar over the air. This document is the end-to-end technical reference for the system: hardware, firmware, on-device AI, the training pipeline, the cloud backend, the dashboard, the communication protocols, and the security model.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Approach	3
1.3	Scope of this document	3
2	System overview	3
2.1	Architecture	3
2.2	Data flow	4
2.3	Multi-user model	4
2.4	Technology summary	4
3	Hardware	5
3.1	Collar	5
3.2	Station	5
3.3	Why split the system this way	6
4	Collar firmware	6
4.1	Responsibilities	6
4.2	Source layout	6
4.3	Flash layout	6
4.4	Build configuration	7
5	Station firmware	7
5.1	Responsibilities	7
5.2	Capture and relay	7
5.3	Robustness: scanning after OTA	8
5.4	Configuration	8
6	On-device AI	8
6.1	Confidence-gated cascade	8
6.2	Matching training and inference	8
6.3	Memory budget	9
6.4	Operator set	9
7	Training pipeline	9
7.1	The loop	9
7.2	Label-agnostic by design	10
7.3	Delivery contract	10
7.4	Known limitation	10
8	Cloud backend	10
8.1	Database schema	10
8.2	Storage	10
8.3	Cloud Functions	11
8.4	Configuration as a trust boundary	11

9	Dashboard	11
9.1	Why a hybrid	11
9.2	Pages and views	11
9.3	Live view and history	12
9.4	Read-only demo	12
10	Communication protocols	12
10.1	Telemetry advertisement	12
10.2	GATT services	12
10.3	Over-the-air model protocol	12
11	Security and privacy	13
11.1	Audio never leaves the collar	13
11.2	Per-owner isolation	13
11.3	The demo is read-only by enforcement	13
11.4	What is and is not a secret	13
11.5	Owner data rights	14
12	Future work	14
13	References	14

1 Introduction

1.1 Motivation

Cats are evolved to conceal illness. A cat that feels unwell will mask it for as long as it can, so by the time a problem is obvious to an owner it is often already advanced. The early signal is rarely a dramatic symptom; it is a quiet shift in routine. A cat that begins to drink noticeably more, eats less, stops grooming, or sleeps through activity it used to enjoy is communicating a change. The difficulty is that no person watches their cat continuously, and day-to-day memory is a poor baseline. The change is real but invisible.

Meowtion makes that change visible. It observes the behaviours that make up a cat's routine, continuously and unobtrusively, and surfaces the trend to the owner. The goal is not to diagnose; it is to notice the routine change early enough that the owner can act, rather than only seeing the crisis.

1.2 Approach

The system is a wearable plus a small fixed gateway. A lightweight, battery-powered collar carries the sensors and the intelligence: it classifies behaviour on the device itself, in real time, and never streams raw sensor data off the cat. It reports only a compact result ("eating, high confidence") over Bluetooth Low Energy (BLE) to a mains-powered base station in the home. The station bridges BLE to the internet over Wi-Fi and forwards results to a cloud backend, which drives a hosted dashboard where the owner sees their cat's activity and trends live.

Two design commitments shape the whole system:

- **Privacy by construction.** Classification happens on the collar. The microphone is used only to help recognise behaviour on-device; raw audio is processed and discarded, never recorded or transmitted. Each owner's data is scoped to their own account by the backend's security rules.
- **Trainable, not fixed-function.** The recognised behaviours are defined by the owner, not baked into firmware. Owners capture and label example clips, a cloud trainer produces tiny models, and those models are delivered back to the collar over the air. Meowtion is a platform for recognising any pet behaviour.

1.3 Scope of this document

This document is the complete technical reference for Meowtion as built. It is organised bottom-up: the physical hardware (section 3), the firmware on each of the two microcontrollers (sections 4 and 5), the on-device AI (section 6), the training pipeline that produces and delivers models (section 7), the cloud backend (section 8), the dashboard (section 9), the communication protocols that tie the tiers together (section 10), and the security and privacy model (section 11). Section 12 records known limitations and planned work.

Section 2 gives the system-level picture first, so the detailed sections that follow have a frame to hang on.

2 System overview

2.1 Architecture

Meowtion is a four-tier system: the collar on the cat, the station in the home, the cloud backend, and the owner's dashboard. Figure 1 shows the data path.

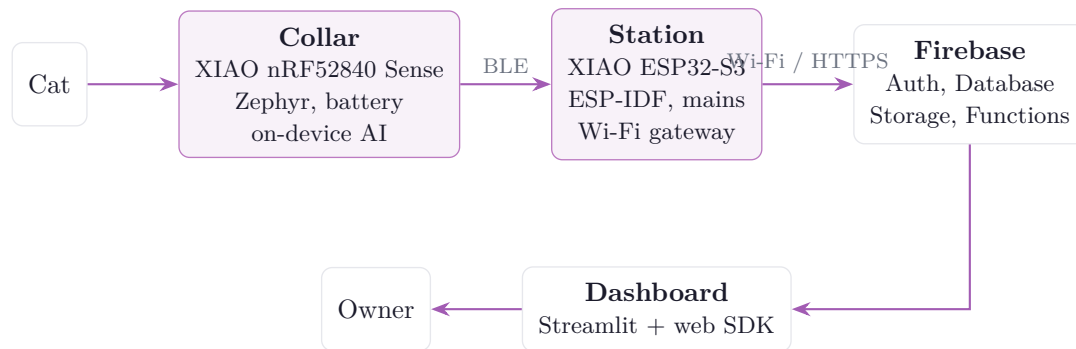


Figure 1: End-to-end data path. The collar is BLE-only, so the always-on station is its gateway to the cloud.

2.2 Data flow

In normal operation the collar runs a classifier on its own sensors and advertises a compact telemetry record over BLE (section 10). The station, scanning continuously, reads that record, attaches the cat and device identity, and writes it to the cloud over an authenticated HTTPS connection. The dashboard reads the same data and renders the owner’s view. Because the collar transmits only a classification result and not raw sensor data, the uplink is tiny and the cat’s audio never leaves the device.

The system has two operating modes, selected per station from the dashboard:

Training mode

The station records short labelled audio clips (with time-aligned motion) while a registered collar is in range, and uploads them for model training. This is how a new behaviour is taught.

Production mode

The station stops recording entirely and simply relays the collar’s on-device classification. This is the everyday monitoring mode.

2.3 Multi-user model

Meowtion is multi-tenant. Each owner creates an account, registers their own station(s) and collar(s), and sees only their own cats. Identity and isolation are enforced by the backend’s authentication and security rules (sections 8 and 11), not by the client. A single public, read-only demo account lets anyone explore the dashboard without sign-in.

2.4 Technology summary

Table 1: Technology at each tier.

Tier	Stack
Collar	Seeed XIAO nRF52840 Sense; Zephyr / nRF Connect SDK; TensorFlow Lite Micro
Station	Seeed XIAO ESP32-S3; ESP-IDF; NimBLE (BLE central) + Wi-Fi
Cloud	Firebase: Authentication, Realtime Database, Cloud Storage, Python Cloud Functions
Dashboard	Streamlit (Python) + Firebase web SDK (JavaScript)
Training	TensorFlow, executed server-side in a Cloud Function

3 Hardware

The build is two boards: the collar worn by the cat and the station plugged in at home. Both are based on Seeed Studio XIAO modules, chosen for their small footprint, integrated radios, and hand-solderable castellated pads.

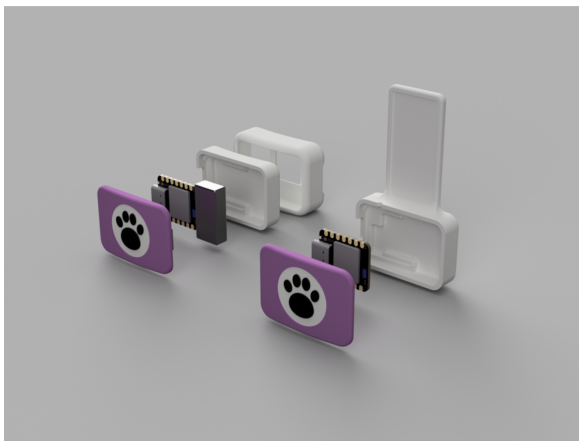
3.1 Collar

The collar is built around the Seeed XIAO nRF52840 Sense. The nRF52840 provides a Bluetooth Low Energy radio and enough flash and RAM to run quantised neural-network inference. Critically for Meowtion, the Sense variant carries the two sensors the system needs on the module itself: a six-axis inertial measurement unit (IMU) and a MEMS microphone, so the collar needs no external sensor breakout.

Table 2: Collar components.

Part	Role
Seeed XIAO nRF52840 Sense	SoC (BLE), IMU and PDM microphone on-module; runs inference
LiPo cell	Battery power for the wearable
Enclosure + strap	3D-printed housing and collar strap (see hardware/)
Acoustic membrane	Protects the microphone port while passing sound

The IMU supplies the motion stream that is always classified; the microphone supplies the short audio window used to disambiguate behaviours that look alike by motion but differ by sound (section 6). The microphone port sits behind an acoustic membrane in the housing; correct membrane placement matters for audio quality and is documented in the hardware guide.



(a) Exploded view of the collar assembly.



(b) Assembled internals.

Figure 2: Collar hardware. Full parts, print files, and assembly steps are in [hardware/README.md](#).

3.2 Station

The station is a Seeed XIAO ESP32-S3. The ESP32-S3 has both Wi-Fi and BLE, so a single module acts as the BLE central that listens to the collar and the Wi-Fi client that reaches the cloud. It is mains-powered over USB, so it can scan and stay online continuously without the power constraints of the wearable. The station carries no sensors of its own; its job is to hear the collar, capture training clips when asked, and bridge to the internet.

3.3 Why split the system this way

The collar must be small, light, and run for a useful time on a small cell, which rules out Wi-Fi on the wearable (its power draw is incompatible with a collar-sized battery). BLE is low-power but short-range and not an internet transport. The fixed, mains-powered station resolves both: it gives the collar a cheap always-on BLE peer and a Wi-Fi path to the cloud. This division also keeps the privacy boundary clean: the heavy, sensitive processing stays on the collar, and only a classification result crosses to the station.

4 Collar firmware

The collar runs on Zephyr via the nRF Connect SDK (NCS v3.3.1), targeting the board `xiao_ble/nrf52840/sense`. It is a BLE peripheral with a USB CDC ACM console for development logging, and it is flashed by drag-and-dropping a UF2 image onto the bootloader drive (double-tap RESET to enter the bootloader). The repository build is driven by `build.ps1`.

4.1 Responsibilities

The collar firmware has four jobs that run from the main loop and BLE callbacks:

1. Sample the IMU (and, when needed, the microphone) and run the classifier.
2. Advertise the result as BLE telemetry for the station to read (section 10).
3. Receive new models over the air and swap them in safely.
4. In training mode, stream sensor data to the station for capture instead of classifying.

4.2 Source layout

Table 3: Key collar source files.

File	Role
<code>main.c</code>	Boot, BLE init, the service loop (model service, then production/sim dispatch)
<code>production.c</code>	Rolling IMU window, runs inference, writes telemetry v2 in production mode
<code>ota.c</code>	OTA receive state machine; commits models and defers the swap to the main thread
<code>classifier.c</code>	Cascade logic with weak hooks for model presence and inference
<code>tflm_classifier.cpp</code>	Strong TensorFlow Lite Micro backend overriding the weak hooks

The classifier is split deliberately. `classifier.c` defines the cascade with *weak* C hooks (model-present queries and inference calls). `tflm_classifier.cpp` provides *strong extern "C"* definitions that override them when the TensorFlow Lite Micro backend is compiled in. This keeps the cascade policy in plain C and the heavy C++ inference cleanly separable.

4.3 Flash layout

Models are not linked into the firmware image; they live in a dedicated flash partition and run execute-in-place (XIP), so a new model can be written without reflashing firmware.

Table 4: Model storage partition.

Partition	Address / size	Contents
models_storage	0xAC000, 0x40000 (256 KiB)	Header plus the IMU (slot 0) and audio (slot 1) model blobs, executed in place

4.4 Build configuration

Enabling on-device inference required pulling TensorFlow Lite Micro into the NCS workspace (it is an optional, excluded west module) and turning on the C++ and TFLM options. The relevant `prj.conf` settings are shown in listing 1.

```

1 CONFIG_CPP=y
2 CONFIG_STD_CPP17=y
3 CONFIG_TENSORFLOW_LITE_MICRO=y
4 CONFIG_TENSORFLOW_LITE_MICRO_CMSIS_NN_KERNELS=y
5 # AllocateTensors()/Invoke() overflow the default ~2 KiB main stack:
6 CONFIG_MAIN_STACK_SIZE=16384
7 # stream_flash lazily erases model pages during OTA:
8 CONFIG_STREAM_FLASH_ERASE=y
```

Listing 1: Inference-related `prj.conf` settings (collar).

Two stability fixes are baked into these settings. The TFLM `AllocateTensors()` and `Invoke()` calls overflow Zephyr’s small default stack, so the main stack is raised to 16 KiB; and because committing a model touches a lot of stack, the swap is deferred off the BLE-RX callback onto the main thread (section 10).

5 Station firmware

The station runs on ESP-IDF. It combines a NimBLE BLE central with a Wi-Fi client, so a single ESP32-S3 both listens to the collar and talks to the cloud. It is built and flashed with `idf.py` over USB serial.

5.1 Responsibilities

1. Scan for registered collars and read their telemetry advertisements.
2. In production mode, relay the collar’s classification to the cloud.
3. In training mode, connect and capture short audio clips (with time-aligned motion) and upload them for labelling and training.
4. Gate, download, and push new models to the collar over the air.

5.2 Capture and relay

When relaying, the station maps the collar’s compact telemetry (mode, class index, confidence) onto the cat’s record in the database, writing both a live “current” state and discrete activity events as episodes begin and end. Events carry the model version and class index so the dashboard can resolve them to human-readable labels even as the label set changes (section 8).

When capturing, the station records audio back-to-back while a registered collar is within a configurable signal-strength gate, uploads each clip to Cloud Storage, and records its metadata in the database for the dashboard’s labelling UI. A manual capture path ignores the range gate,

which is used to collect behaviours such as purring that happen while the cat rests rather than at the bowl.

5.3 Robustness: scanning after OTA

The station uses one BLE radio for several jobs, and OTA push exposed a subtle defect. To push a model the station must cancel scanning and open a connection to the collar; the OTA path used its own GAP callback and never restarted scanning when it finished, so after an update the station went deaf and the dashboard reported the collar offline. The fix is an idempotent `ble_resume_scan()` (guarded on the discovery-active flag) that the OTA path calls on both completion and bail-out. This was verified on hardware.

5.4 Configuration

Wi-Fi credentials are provisioned to the station over USB during device pairing in the dashboard and are never stored in the repository (see the `.gitignore` note on `secrets.h`). The capture range (signal threshold and dwell) and the operating mode are configured per station from the dashboard and read by the firmware from the database.

6 On-device AI

All classification happens on the collar, in real time, using TensorFlow Lite Micro (TFLM) [1] with the CMSIS-NN kernels [2] for the nRF52840's Cortex-M4F. The models are tiny int8 networks in the spirit of the TinyML approach [3].

6.1 Confidence-gated cascade

Two behaviours can look almost identical to a motion sensor while sounding completely different: eating and drinking both involve the head down at a bowl. Rather than force a single model to separate them from motion alone, Meowtion runs a confidence-gated cascade of two models:

1. The **IMU model** runs continuously on the motion stream. It is cheap and always on.
2. When the IMU model is *unsure*, a short **audio model** is invoked to confirm. Audio is therefore a deployed signal, used to resolve ambiguity, not merely a training aid.

This keeps average power low (the audio path runs only when needed) while recovering the accuracy that motion alone cannot provide.

6.2 Matching training and inference

The single most important correctness property is that the on-device input transform must mirror the transform used during training, exactly. A model trained on one representation and fed another at inference will produce confident nonsense. Meowtion enforces this on both paths:

- **Motion.** The rolling window is normalised per channel: subtract each channel's mean over time, then divide by the global maximum absolute value (with a small epsilon), and quantise to int8 using the input tensor's own scale and zero-point.
- **Audio.** Inference audio is decimated and encoded to match the training representation: 8 kHz μ -law, produced by the same conversion used to prepare training clips. A mismatch here was an early source of confident misclassification.

To avoid a large floating-point scratch buffer on a RAM-constrained part, the IMU path quantises directly from the int16 source in a small number of passes (compute the per-channel mean and the global max, then quantise), rather than materialising a float window. Listing 2 sketches the approach.

```

1 // pass 1: per-channel mean over the window
2 // pass 2: global max-abs of (sample - channel_mean)
3 // pass 3: write int8 = round((x_norm)/scale) + zero_point
4 //           where x_norm = (sample - channel_mean) / (maxabs + 1e-6)
5 // scale and zero_point come from the input tensor itself.
```

Listing 2: Direct int16-to-int8 quantisation, mirroring training (sketch).

6.3 Memory budget

TFLM uses a fixed tensor arena per model. The arenas are sized to the two networks and are the dominant RAM cost of inference:

Table 5: Tensor arenas.

Model	Arena	Notes
IMU (slot 0)	16 KiB	Always-on motion classifier
Audio (slot 1)	48 KiB	Invoked only to confirm an uncertain IMU result

Enabling inference initially pushed RAM use to roughly 94%; eliminating the float window scratch buffer (quantising directly from int16) brought it back to the low 80s, leaving headroom for the BLE stack and the raised main stack.

6.4 Operator set

The op resolver is a fixed `MicroMutableOpResolver`, so only the operators the models actually use are linked. The Keras `Conv1D` layers are exported as `CONV_2D` wrapped in `EXPAND_DIMS` and `SQUEEZE`; the missing `EXPAND_DIMS` operator was the cause of an early “didn’t find op” load failure, and the resolver now includes it along with the pooling, fully-connected, reshape, softmax, and quantise/dequantise operators the networks need.

7 Training pipeline

Meowtion learns new behaviours through a closed loop that an owner drives entirely from the dashboard: capture, label, train, deliver, run. No local toolchain is required.

7.1 The loop

1. **Define behaviours.** The owner lists the actions to recognise (eat, drink, purr, scratch, litter-tray, and so on) in the dashboard. This set drives both the labelling UI and the trained model’s outputs.
2. **Capture.** With a station in training mode, short audio clips and time-aligned motion are recorded while a registered collar is in range, and uploaded to Cloud Storage.
3. **Label.** The owner reviews clips in the dashboard, sets each clip’s action, trims dead space, and discards poor examples.

4. **Train.** A Cloud Function trains the model(s) server-side from the labelled clips and writes the results back: the quantised `.tflite` blobs to Storage and the version and label list to the database.
5. **Deliver.** Each station watches a model-version key. When it changes, the station downloads the new `.tflite` files and pushes them to the collar over the air (section 10).
6. **Run.** The owner switches the station to production mode; the collar classifies with the new model and the dashboard shows live, named behaviours.

7.2 Label-agnostic by design

The trainer is not hard-coded to a fixed set of actions. It learns whatever label set the owner defines, which is what makes Meowtion a platform rather than a fixed-function gadget. The same pipeline that recognises eating and drinking can be pointed at any distinguishable behaviour the owner can capture and label.

7.3 Delivery contract

Training outputs are written to predictable locations so the station can find them without coordination: the model blobs land in Storage under the owner's path as `imu_model.tflite` and `audio_model.tflite`, and the new version is mirrored into each of that owner's stations' configuration so the station's gate fires. The label list is stored alongside so the dashboard can map a class index back to a name (section 8).

7.4 Known limitation

A two-class model (for example, eat versus drink only) has no way to represent “neither” and will confidently assign one of its two labels to an idle cat. The fix is to retrain with an explicit idle/other class; this is a data and modelling task, not a firmware change, and is tracked in section 12.

8 Cloud backend

The backend is Firebase. It provides authentication, a Realtime Database for live state and history, Cloud Storage for training clips and model blobs, and Python Cloud Functions for server-side training. There is no custom server to operate.

8.1 Database schema

State and history live in the Realtime Database, keyed by owner so the security rules can scope every read and write to the owning account. Table 6 lists the main paths.

Events store the model version and class index rather than a pre-resolved name. The dashboard maps the index to a label using the version's label list, so historical events stay correct even as the owner changes their behaviour set.

8.2 Storage

Cloud Storage holds the training clips (audio plus aligned motion) and the trained model blobs. Training clips are private to the owner and are deliberately *not* world-readable, even in the demo: home audio must never be public. The model blobs are what the station downloads during over-the-air delivery.

Table 6: Realtime Database paths (illustrative).

Path	Contents
users/<uid>/devices/<token>/	A registered station or collar, by scoped token
.../cats/<catId>/current	The cat’s live state (version, class, confidence)
.../cats/<catId>/events	Discrete activity episodes (carry version + class index)
users/<uid>/models/	Current model version and label list for the owner
config/demoOwner	UID whose data backs the public read-only demo
config/modelVer	Per-station model-version key the station watches
config/devAccounts/<uid>	Flags accounts allowed to use the dev/training tools

8.3 Cloud Functions

A Python Cloud Function performs training. It reads the owner’s labelled clips, trains the model(s) with TensorFlow, writes the quantised `.tflite` outputs to Storage, records the new version and labels in the database, and mirrors the version into each of the owner’s stations so delivery is triggered (section 7). Running training server-side keeps the owner’s experience to a single button and removes any local setup.

8.4 Configuration as a trust boundary

The `config` subtree is readable by clients but not writable by them; it is maintained through the console or an administrator path. This lets the system publish facts the clients need (which account backs the demo, the current model version, which accounts are developers) without allowing a client to grant itself privileges.

9 Dashboard

The owner-facing application is a Streamlit dashboard backed by a small set of static pages for the parts that need the Firebase web SDK directly (sign-in, account and device management, the developer/training console, and the privacy policy). It is hosted on Streamlit Community Cloud and deploys automatically from the repository’s main branch.

9.1 Why a hybrid

Authentication and device pairing are easiest with the Firebase web SDK in the browser, while the data views and charts are easiest in Streamlit’s Python. The two halves are served from the same origin so they can share a login: the static sign-in page writes the Firebase ID token into a same-origin cookie, and the Python dashboard reads that cookie to identify the owner. A separate flag cookie marks the public read-only demo.

9.2 Pages and views

Table 7: Dashboard surfaces.

Surface	Role
app.py (Streamlit)	Live cat state, activity history, charts, account controls
account.html	Sign-in, account, device pairing over Web Serial
dev.html	Developer/training console: capture, label, train, mode
privacy.html	Privacy policy

9.3 Live view and history

The dashboard reads the cat’s live “current” record and renders the latest behaviour, and it flattens the event history into a table for charts (totals per behaviour, frequency, and trends over time). Activity charts are rendered with a transparent background so they sit on the page’s brand canvas rather than in a clashing panel.

9.4 Read-only demo

A public demo lets anyone explore the product without an account. It reads the demo owner’s data (permitted by the security rules) but every write control and function trigger is disabled, and the developer/training console is available in a read-only variant. The demo is genuinely read-only because it is enforced by the backend rules, not just hidden in the UI: the demo account is world-readable but write-locked (section 11).

10 Communication protocols

Three contracts tie the tiers together: the BLE telemetry the collar advertises, the GATT services for streaming and updates, and the over-the-air model protocol. Keeping these explicit is what lets the collar and station firmware evolve independently.

10.1 Telemetry advertisement

In production mode the collar publishes its result in the manufacturer-specific field of its BLE advertisement, so the station can read it passively without connecting. Version 2 of the contract is shown in table 8.

Table 8: Telemetry v2 manufacturer-data contract.

Byte	Field	Meaning
mfg[2]	mode	2 = real (on-device classification), 1 = simulated
mfg[3]	class index	Predicted class, or 0xFF for unknown/uncertain
mfg[4]	confidence	Confidence, 0–100

Because the class is an index, not a name, the station and dashboard resolve it through the label list for the running model version, keeping history correct across label-set changes (section 8).

10.2 GATT services

For training-mode streaming and for model updates the station connects to the collar. Streaming carries audio and motion to the station for capture; the over-the-air service carries model data to the collar. The OTA service uses a small family of 0x0b** UUIDs and a status notification the collar uses to acknowledge progress and report success or failure.

10.3 Over-the-air model protocol

Model delivery is a simple, framed protocol shared by both firmwares through a common header. A transfer is BEGIN, a stream of DATA chunks, then END; the BEGIN frame declares the target slot, the model length, and a CRC so the collar can reject a corrupt transfer before committing it.

```
1 enum meow_ota_op    { MEOW_OTA_BEGIN = 1, MEOW_OTA_DATA = 2, MEOW_OTA_END = 3 };
2 enum meow_ota_slot { MEOW_SLOT_IMU   = 0, MEOW_SLOT_AUDIO = 1 };
```

```
3
4 struct meow_ota_begin {
5     uint8_t op;           // MEOW_OTA_BEGIN
6     uint8_t slot;         // MEOW_SLOT_IMU | MEOW_SLOT_AUDIO
7     uint32_t len;         // model size in bytes
8     uint32_t crc32;       // CRC-32/IEEE over the model blob
9 } __packed;
```

Listing 3: Shared OTA framing (`firmware/common/meow_ota.h`, illustrative).

On the collar, END verifies the CRC, writes the model header, and sets a pending flag; it does *not* swap the live model from inside the BLE callback. The actual swap is performed later by a service call from the main loop, on the main thread’s larger stack, because committing a model is stack-heavy and doing it in the radio callback crash-looped the device. This split, together with the raised main stack (section 4), is what made OTA reliable on hardware.

11 Security and privacy

Privacy is a design property of Meowtion, not a setting. The two strongest guarantees come from where work happens and how data is scoped.

11.1 Audio never leaves the collar

The microphone is used only for on-device classification. Audio is processed on the collar and discarded; it is never recorded to flash for normal operation and never transmitted in production mode. The collar’s uplink is a classification result (a class index and a confidence), not a sensor stream. The one exception is deliberate and owner-driven: in training mode the owner chooses to capture clips to teach a behaviour, and those clips are stored privately to their own account.

11.2 Per-owner isolation

Every read and write is scoped to the owning account by the database and storage security rules. A device authenticates with a scoped, revocable token rather than the owner’s password, so a compromised or lost device cannot expose credentials and can be revoked without a password change. Training clips and home audio are private to the owner and are not world-readable, even for the public demo.

11.3 The demo is read-only by enforcement

The public demo reads a single designated demo account’s data, which the rules make world-readable, but that account is write-locked at the backend: the demo cannot change any data or trigger any function. The read-only behaviour is enforced by the rules, not merely hidden in the interface, so it holds even against a crafted client.

11.4 What is and is not a secret

The Firebase web API key shipped in the static front end is a public client identifier, not a secret; security is enforced by Authentication and the rules, not by hiding the key. No service-account key is present in the repository, and device Wi-Fi credentials and firmware secrets are excluded by `.gitignore`. Everything required to understand and build the system is tracked; only true secrets are not.

11.5 Owner data rights

The account page lets an owner export everything stored about their cats and delete their account, devices, and data. This supports the access and erasure expectations of data-protection regimes such as the UK GDPR; the bundled privacy policy is a starting point and is marked for legal review before any public launch.

12 Future work

The system works end to end on real hardware: record, label, train, deliver over the air, classify on-device, and display live. The following items are known and planned.

Idle/other class

Retrain with an explicit “neither” class so an idle cat is not forced into one of the trained behaviours. This is a data and modelling task (section 7).

Activate the audio confirm stage

The cascade is built for IMU-then-audio confirmation; finish wiring the audio second stage into production so ambiguous motion is resolved by sound on-device (section 6).

Battery characterisation

Measure real collar runtime per behaviour mix and tune the duty cycle of the audio path against it.

More behaviours

Extend the captured and labelled set beyond eat/drink/purr to scratching, litter-tray use, grooming, and play.

Multi-cat disambiguation

Distinguish multiple cats in one household, currently framed as a single-cat, within-subject problem.

Firmware OTA

Today only models are delivered over the air; extend the mechanism to full firmware updates.

Publication

Write up the confidence-gated cascade as a short preprint and a venue paper.

13 References

- [1] Robert David et al. ‘TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems’. In: *Proceedings of Machine Learning and Systems (MLSys)* 3 (2021). arXiv: [2010.08678](#).
- [2] Liangzhen Lai, Naveen Suda and Vikas Chandra. ‘CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs’. In: *arXiv preprint* (2018). arXiv: [1801.06601](#).
- [3] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019. ISBN: 9781492052043.